

TM252: Block 3 Activities

Contents

Week 19 - Architecture, Application and Languages

- The Python Language
- Commenting in Python
- Python Variables
- Python Data Structures
- Python Conditionals
- Iteration in Python
- Web server overview
- A Simple Python Web Server
- Week 19 - Deeper Exploration

Week 20 - Advanced Web Applications in Python

- Introduction to Web Frameworks
- An introduction to Flask
- Implementing an Echo Server
- Handling Parameters
- Request Handling (GET and POST)
- Handling Complex Data with JSON
- Session Handling
- Persistent Data Storage
- Web Services
- RESTful services
- Week 20 - Further Exploration
- Security in Web Services

Miscellaneous

- Errata
- Accessibility Statement

Welcome to the practical, programming-related activities for TM252 Web Technologies, Block 3.

Environment Overview

Throughout the materials in Week 19, we have discussed a number of different concepts in relation to the components of a web-server. In addition, we have explored the ways in which we can instruct the server to perform the functions we require through an application using a programming language; Python.

The activities contained within this workbook will help you to familiarise yourself with the fundamentals of this language before helping you develop the necessary skills to implement a functional web-server application.

To implement a python script you will need access to an editor and an interpreter. In previous weeks of this module (weeks 10-13), you have utilised the OpenComputing Lab to test practical elements of web development. This same approach will be used in the completion of the activities for this week. You will use the development environment provided by the OCL to implement and complete the example scripts outlined here.

Activity: Source File Manipulation

The first task we will undertake is a review of the creation, modification and removal of a file in the OCL environment.

1. Go to the Module Website in a separate browser tab or window and from there access the OpenComputing Lab (OCL). You can find the link to access the OCL in the **Resources** section or in each week in the planner.
2. In the OCL click on the 25B **TM252 Block 3** VCE.
3. Create a document and name it "main.py 
4. Save the document "main.py 
5. Delete the file you just created.

Content in the VCE

When the VCE loads, you should create a folder that is memorable to yourself such that you can locate the scripts that you create.

When you have created your folder with title '<your_folder_name>', you may store your first script here

Much like the previous activities undertaken within this module (weeks 10-13), you will notice a number of folders within your workspace. These folders work in exactly the same way as discussed in previous sections, but here are the important points:

- **skeletons**: Contains skeleton folders for the activities in Block 3 and TMA03.
- **tma03**: Working area for TMA03
- **work**: Working area for the Block 3 activities.

When you start an activity, you will generally be instructed to copy a folder from the **skeletons** folder into the **work** folder and then work inside that copy. If at any point during an activity you feel you have gotten completely stuck and see no way to get it to work, then you can always delete your folder from the **work** folder and copy it from the **skeletons** folder again.

Warning

Do not edit any of the files in the `skeletons` folder. These will automatically be overwritten every time you start the VCE.

Commenting in Python

Programming instructions are written in languages that can be understood by a computer when coupled with development tools and the right environment. These same instructions are not always quite so readily understood by software developers, sometimes even the author of the collection of instructions if enough time has passed.

Fortunately, to counter this problem, most programming languages allow for the addition of commentary for the instructions included within a source file. The Python programming language is no exception to this, offering many options when including notes about the intention of the code.

Comments are used to explain what your code is doing, making it easier to understand for both you and others. Comments are not executed by Python—they're just there for humans. They are very helpful for debugging, collaboration, and improving code readability.

Types of Comments in Python

Single-Line Comments These comments are written on a single line and start with the `#` symbol. Everything after the `#` is considered a comment and is ignored by Python.

```
# This is a single-line comment
a = 5 # This is an in-line comment
```

Multi-Line Comments

You can use multi-line comments to explain things that take more than one line.

You have two options for multi-line comments: Using multiple `#` symbols, one for each line. Using triple quotes (`'''` or `"""`) for a block of text. While this is technically used for docstrings, it can also serve as a comment.

```
# This is a multi-line comment
# that explains something important
# over several lines.
```

```
'''
This is a multi-line comment
that uses triple quotes.
It can span across many lines.
'''
```

You can comment out parts of the code to test changes without deleting them. This can often help with debugging your code, omitting execution whilst retaining the instructions. This approach can sometimes lead to issues where code is misinterpreted by the interpreter so should be used carefully.

```
'''  
a=5  
print(a)  
'''
```

Activity: Create a python script file with commentary

Go to the Module Website in a separate browser tab or window and from there access the OpenComputing Lab (OCL). You can find the link to access the OCL in the Resources section or in each week in the planner.

In the OCL click on the 25B TM252 Block 3 VCE.

1. Create a document and name it "script_comments.py"
2. Save the document "script_comments.py"
3. Add a single-line comment to the script to indicate the file name
4. Add a multi-line comment to denote your PID ad the module name and presentation through a series of single lines
5. Add a multi-line comment to this file using the mult-line syntax that indicates the reason you have chosen to study this module
6. Save this script for later reference

Python variables

Variables are editable areas of memory to which we can assign a label so that we may use its stored data later within a program. They are a fundamental concept in programming and are used in all languages for data storage and manipulation.

To create a variable in python we simply need to state the name of the storage location, which creates the variable and assign it a value.

```
daffodilCount=25  
genus="Narcissus"  
lConstant=6.3
```

Python handles variables dynamically, meaning we do not need to declare their type explicitly. Dynamic typing means that we do not need to let the computer know that we will store whole numbers, textual or raw byte-data in the specified location. In the following example, we have assigned the numeric value of 25 to the `daffodilCount` variable and later assigned the text value "25" to the `daffodilCount` variable.

```
daffodilCount=25  
daffodilCount="25"
```

Whilst Python is dynamically typed it is possible to add "hints" to the interpreter relating to the type of information you may want to store in a variable with type annotations. An example of this can be seen below with the `daffodilCount` variable being declared with integer data type and the `genus` variable being declared with a string data type

29.
var
na
da

```
daffodilCount: int = 10
genus: str = "Narcissus"
heightMetres: float = 1.25
```

There are a number of rules governing the names that may be used for variables. These typically preclude the specific formats and keywords from use as they would interfere with operations in Python.

For conciseness you may declare and assign multiple variables in one line of code. How you declare variables can often be determined by preference and style. Though some people may find this a more elegant solution it can sometimes lead to difficulty in determining where variables are initialised, especially when starting to learn the Python language.

```
a, b, c = 1, 2, 3
```

Warning

Naming Rules

1. Variable names must start with a letter (a–z, A–Z) or an underscore (_).
2. The rest of the name can include letters, numbers (0–9), and underscores.
3. Variable names are case-sensitive (myVar and myvar are different).
4. You cannot use Python's reserved keywords (e.g., if, for, while, class) as variable names.

To output the value of the data stored in a variable we can use a simple instruction in Python; the `print()` function. The `print()` function allows us to output something to the screen so that we can receive feedback from our script. This function may be used in the following way:

```
message="hello"
messageLetters=5

print("hello")

print(message)
print(messageLetters)
```

Note how the print function may take a literal string and print this to the screen, or it may take the contents of a variable and print this to the screen in the same way.

To make a script more flexible we may need to input data whilst the programming is running. We can achieve this through the use of the `input()` function. The input function blocks the script from executing further instructions while it waits for the user to provide input. This is useful as it allows the input of data that may not be known before the program executes. An example of how this can be achieved is given below whereby we request input and assign this input value to a variable.

```
inputData=0
inputData = int(input("Please input a number between 0 and 9"))
print(f"input value is {inputData}")
```

Note

The above script introduces quite a number of new concepts including taking input from the user. On the first line of this script, we see the `inputData` variable being declared and initialised with the number 0.

The second line of the script is a little more complicated. The first thing to note is that we are assigning a value to this variable with the use of the '=' operator.

On the right side of this statement, we have the `input()` function, only we have given it an argument much like we would have for the `print()` function. Providing this text to the function means that Python will print the text to the screen and wait for the input by the user.

The last important point here is that the `input()` function is enclosed within a set of brackets. When we enter data into the console it is done so in a text format. We cannot compare data of two different types and expect the Python to treat them the same. This is one instance where dynamic typing can cause issues. To ensure we can store data in the way we want, we sometimes need to convert the input to that data type; this is the purpose of the `int()` enclosing the input data that is assigned to the variable.

Activity: Variable Practise

Go to the Module Website in a separate browser tab or window and from there access the OpenComputing Lab (OCL). You can find the link to access the OCL in the Resources section or in each week in the planner.

Having explored the ways in which variables may be declared and assigned, complete the following activity.

In the OCL click on the 25B TM252 Block 3 VCE.

1. Create a document and name it "script_variables.py"
2. Save the document "script_variables.py"
3. Add comment to the file to indicate the name and date
4. create a variable with the name 'pid'
5. Assign the 'pid' variable the value of your pid (this can be numeric or string)
6. Create a variable to store your name as a string.
7. Create a variable to store the length of time of this activity (30 mins), in hours, as a float.
8. Print the contents of each of the variables to the screen
9. Save the script for future reference

Python Data Structures

Sometimes it is necessary to store data elements in addition to the relationship it has with other components of data. Whilst the use of variables is beneficial to store individual values, it is also possible to store values in a way that denotes relationships between them through the use of simple data structures like arrays.

An array can be considered a contiguous block of memory that we declare and assign values to in a similar way to that seen with variables.

In Python, arrays are commonly represented using lists or the array module. Arrays are used to store multiple values in a single variable, making it easier to organize and work with large amounts of data.

Note

Key Features of Arrays:

1. Ordered - Items in an array have a specific order.
2. Indexing - Each item can be accessed using its position, starting from index 0.
3. Mutable - The values in an array can be changed.

To use a list as an array in python, we can declare this by first providing the label name we wish to give to this storage location, much like we did with variables, followed by the values we want to store there. This is slightly different to variable assignment as we now have the concept of indexes within the storage location.

```
animals = ["horse", "iguana", "cat"]
```

This statement creates a large storage location that we have placed the values `horse`, `iguana` and `cat` into. When we want to retrieve this data we need to tell the script what part of the memory location to get a value from. This is achieved by providing the index within the location we wish to fetch data from.

```
animals = ["horse", "iguana", "cat"]  
print(animals[1])
```

Using the same `print()` function as we used in our variable examples, we can output the value stored in a particular part of the array by providing the index number of the item. The output to the above code would be `iguana` as a result of providing the index 1.

In most contexts, computers store information using a 0-based numbering system, meaning that the first number in a sequence will be 0, therefore the first location of the array will be element 0. If we wanted to output the second element in the array, we would use the number 1 and increment this for consecutive values.

We can also change the values stored in any index of the array by treating it like an individual variable where we also provide the index. This means that in the same way as we have changed the value of variables, we can use a very similar syntax to perform the same function on an array value; this is shown in the below code.

```
animals = ["horse", "iguana", "cat"]  
print(animals[1])  
animals[1] = "lemur"  
print(animals[1])
```

The first `print` statement for this script will output the word `iguana` whereas the second print statement will output the word `lemur` following the amendment.

we can add items to the array using an inbuilt function of the Python programming language; `append()`. The `append()` function allows for the addition of an item to the array and is placed immediately after the last item already contained within the list. The usage of this function can be seen below where `fish` is added to the list.

```
animals = ["horse", "iguana", "cat"]
animals.append("fish")
```

Note that the `append()` function is a function of the array itself. This means that the movement is handled by the storage location and not something that needs to be managed by yourself. This is a beneficial abstraction of Python that makes many operations far more accessible than other languages. We can now access the array at the new index to output the new value we have added.

A similar operation to the `append()` function may be used to remove an element from the array in the form of the `remove` function. There is one caveat here in that when using this function, we do so by specifying the value of the item we wish to remove and not the index. To remove items by index we would use the `del` function; both of which can be observed below.

```
animals = ["horse", "iguana", "cat"]
animals.append("fish")
animals.remove("horse")
del animals[1]
```

To determine the output of this script we need to consider the operations being executed here. The first instruction declares and initialises the array with 3 values. The second instruction adds the element `fish` to the array. The third instruction removes the element `horse` from the array leaving `iguana`, `cat` and `fish`. When we attempt to remove the element at index of 1 with the `del` function, this means we then lose the `cat` element, leaving us with two remaining data elements; `iguana` and `fish`.

Activity: Array Practise

Go to the Module Website in a separate browser tab or window and from there access the OpenComputing Lab (OCL). You can find the link to access the OCL in the Resources section or in each week in the planner.

Having explored the ways in which common data-structures may be declared and assigned, complete the following activity.

In the OCL click on the 25B TM252 Block 3 VCE.

1. Create a document and name it "script_data_structures.py"
2. Save the document "script_data_structures.py"
3. Add a comment to the file to indicate the name and date for the file
4. Create an Array with a list of fruits for its initial value (["apple", "banana", "cherry", "date"]).
5. Add "orange" to the basket using the 'append()' method.
6. Print the name of the second fruit in the list.
7. Replace "banana" with "blueberry".
8. Print the contents of each of the variables to the screen
9. Save the script for future reference

Python Conditionals

Computer programs typically execute instructions in the order in which they are written. In some circumstances it is necessary to change the order of execution or not execute instructions at all. To do so we employ the use of conditional or branching statements.

Conditionals in Python are a way for your program to make decisions. They let you execute certain code only if a specific condition is true. This can be considered the same as asking a question which provided a true or false answer; does Python output text to the screen? Are you studying TM252? Is it raining today?

We can implement conditionals with the use of the `if` keyword. The `if` keyword presents Python with an opportunity to determine whether something is true or false. The typical use of this can be seen in the script example below.

```
x=5
if x > 4:
    print("x is greater than 4")
```

Warning

This is where the Python syntax becomes a little more complex.

Take another look at the example use of the `if` statement here and you will notice there are a number of differences in this script to what we have previously seen.

The first point of note is the use of `:` at the end of the `if` instruction. This symbol is used to denote a code block in where we tell the interpreter that the next set of instructions should happen only if the condition is true.

The second thing you'll notice is the indentation of the following instructions found on the line after the `if` statement. Statement placement is important in Python as the interpreter uses this to identify relationships between instructions; in this case, the `print()` statement is executed only if the result of the `if` statement is true.

The use of the `if` statement presents us with a means of conditionally executing instructions but is often used in conjunction with another keyword; the `else` statement. The `else` keyword is an operator in Python that allows for specifying whether your script should execute one collection of instructions or another depending on whether a condition meets one criterion or another.

```
x=3
if x > 4:
    print("x is greater than 4")
else:
    print("x is less than 4")
```

In the example script provided above we have changed the value of the variable and are now providing instructions to execute in the event that the condition is satisfied and a provision for if the condition is not met. It is important to note the indentation here with respect to statements that follow keywords as this denotes their relation to this construct.

Activity: Conditionals Practise

Go to the Module Website in a separate browser tab or window and from there access the OpenComputing Lab (OCL). You can find the link to access the OCL in the Resources section or in each week in the planner.

Having explored the ways in which conditionals may be employed, complete the following activity.

In the OCL click on the 25B TM252 Block 3 VCE.

1. Create a document and name it "script_conditionals.py"
2. Save the document "script_conditionals.py"
3. Add a comment to the file to indicate the name and date for the file
4. Add a statement to declare a variable called "secret" assigning this the value of "secretMessage"
5. Add a statement that creates another variable called "testValue" and give it an empty string value
6. Add a statement that requests input from the user requesting a password and assign this to the "testValue" variable
7. Add a conditional statement to compare the "testValue" and "secret" variable, if these match output "Access Granted". If these values don't match provide your own error message and print this to the screen.
8. Save the script for future reference

Iteration in Python

When writing a computer program, we often need to perform a single instruction multiple times. We could achieve this by writing the same instruction in our scripts the number of times we need to execute them, but this could prove unwieldy if we need to perform an operation some great magnitude of times. In addition, this does not account for uses cases where we may want to perform an instruction or a group of instructions indefinitely. To solve this problem, we use iteration.

Iteration in Python is the process of repeating a set of actions, usually over a collection of items like a list, string, or range of numbers. It's a key part of programming because it helps automate repetitive tasks. There are a number of ways in which we can employ the concept of iteration all of which depend on the context where we may want to repeat our steps.

The simplest of these, and perhaps the most frequently used, is the `for` loop. This approach to iteration is used in the event that we know how many times we require the repetition. An example of this may be seen below. Where we have an array of fruits with three items contained within. We then follow this with the `for` loop creating a temporary variable named `fruit` that we use to iterate through the items in the array. In the same manner as we saw with conditionals, we may show relationships between keyword actions and instructions with ':' and the use of indentation.

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

An alternative to iterating through the values in a list exists for the `for` loop where a further function may be used to define a number of times to repeat the instruction; the `range()` function. The `range()` function is used to create a collection of numbers up to a given value; an example of this is used can be seen in the code below.

```
for count in range(5):  
    print(count)
```

In the above script, we use the `for` keyword to denote that we want to start a loop. In this statement we also create a temporary variable `count` for the loop. We then see the `range()` function, where we have provided the number 5 to limit the number collection generated by this function. This means that the loop will output the value of the temporary variable 5 times here.

There may be instances where we do not necessarily know the number of times we would like to repeat instruction, instead opting to repeat until a given condition has been met. This incorporates the concepts learned in the conditionals section and iteration. To implement this type of loop, we can use the `while` keyword.

```
count = 0  
while count < 3:  
    print(count)  
    count += 1
```

In the above example, we assign a variable in which we may store the current iteration count. We then use the `while` keyword to denote we would like to repeat the related instructions until a condition is met. In this case, the condition that we are hoping to meet is that the value of the `count` variable will equate to 3. Within the loop, we are printing out the value of the `count` variable and then incrementing this value by one for each new iteration. When the value of `count` meets the condition check, the loop is terminated.

Activity: Iteration Practise

Go to the Module Website in a separate browser tab or window and from there access the OpenComputing Lab (OCL). You can find the link to access the OCL in the Resources section or in each week in the planner.

Having explored the ways in which iteration may be employed, complete the following activity.

In the OCL click on the 25B TM252 Block 3 VCE.

1. Create a document and name it "script_iteration.py"
2. Save the document "script_iteration.py"
3. Add a comment to the file to indicate the name and date for the file
4. Add an instruction to print the words `hello world` to the screen 5 times
5. Add a variable to hold a numeric value, assign this variable the value of 7
6. Add a second variable to hold a numeric value, assign this variable the value of 0
7. Add an instruction to request a number as input and assign this to the first variable created
8. Add an instruction to increment the first variable using a `iterator` with a range equal to that of the the input value collected from step 7.
9. Save the script for future reference

Web-Server Overview

We have already seen that a web-server is composed of a consortium of disparate parts all working together to support connection to and from web resources. For week 19 we have primarily focused upon developing Python programming skills so that we are able to create functionality for a web-application.

Now that we have a basic understanding of the building blocks used in the construction of a web-application, we will work on setting up a web-server to test some of these building blocks a little further. Rather than implement all of the functionality of a web-server, we will leverage the functionality built into our programming language.

For the most basic version of this, we have already seen that we can start a web-server instance by issuing a console command instructing an installed Python interpreter to do so with very few arguments.

This section however, will focus on the skills discussed in week 19 related to the development of a basic web-server in Python with a little more customisation than is offered by the pre-built module.

A Simple Python Web Server

This simple web server demonstrates the core principles of how web applications work. It listens for incoming requests, processes them, and sends back responses—just like any real web server. By using only Python's built-in modules, it avoids unnecessary complexity, making it easier to understand the fundamentals.

At its core, this implementation highlights key web server concepts: it continuously listens for requests, processes GET requests and responds with a message. It also demonstrates important ideas like port binding, HTTP's stateless nature, and the client-server model. These are the same principles that power real websites and APIs, just on a much larger scale.

By learning how a basic server functions, you build a strong foundation for more advanced web development. Whether you move on to frameworks like Flask or Django, or dive deeper into networking, this understanding will help you develop and troubleshoot web applications more effectively.

```

from http.server import HTTPServer, BaseHTTPRequestHandler #imports the functionality required to run th

#creates a function to handle a request to the default address
def handle_request(self):
    self.send_response(200) # provides an OK http response
    self.send_header("Content-type", "text/plain") # determines the content type to return
    self.end_headers() #concludes header sending
    self.wfile.write(b'Hello, World!') # writes the content to send

#creates a function to handle a request to a second address endpoint
def handle_new_message(self):
    self.send_response(200)
    self.send_header("Content-type", "text/plain")
    self.end_headers()
    self.wfile.write(b'This is a new message!')

#defines and extension class to manage http functionality
class SimpleHandler(BaseHTTPRequestHandler):
    def do_GET(self): # function called whenever a request is received
        if self.path == '/': #this is the default path to the server, calls the default response function
            handle_request(self)
        elif self.path == '/new_message': # handles a second endpoint and calls the handle_new_message()
            handle_new_message(self)
        else:
            self.send_response(404) # in the event this server doesnt know the address added, a not found
            self.end_headers()
            self.wfile.write(b'Not Found')

#function to perform set up the server and allow incoming connections
def serve():
    server = HTTPServer(('localhost', 8000), SimpleHandler)
    print("Serving on port 8000...")
    server.serve_forever()

if __name__ == '__main__':
    serve()

```

There are a number of different concepts introduced within this script that we have not seen before. The following provides an overview of the instructions within the script to give you an idea of what each component does.

Imports:

```

HTTPServer from http.server: Used to create the web server.
BaseHTTPRequestHandler from http.server: Typically used for handling HTTP requests, but here it's used i

```

Functions:

```

handle_request(request, response)
    Sends a 200 OK response with the content type text/plain.
    Writes "Hello, World!" as the response body.

serve()
    Defines a request handler function (handler) that initializes a BaseHTTPRequestHandler instance.
    Overrides do_GET with handle_request.
    Starts an HTTPServer on localhost:8000 using this handler.
    Runs indefinitely, serving requests.

```

Note

Observing the output of web applications using the VCE is a little different from running your Python script and seeing the output in the console. To see the visual output of your endpoints, perform the following steps:

1. Ensure that you have saved your script file in an appropriate directory; make note of this as this will be used to run the script.
2. Ensure that you have the terminal open in the VCE
3. Enter the following command in the terminal replacing `<path to your file>` with the actual path noted in step 1 here. If you are comfortable with directory navigation in bash you may wish to navigate to the appropriate directory to work with shorted paths.

```
python3 ./<path to your file>
```

4. Once you have executed this command, the terminal should respond with the string:

```
Serving on port 8000...
```

5. The VCE will also notify you that the server has started with a notification box in the bottom right of the screen. An option to view the output of this in the browser is provided. Clicking `open browser` will navigate to the served content
6. you can also access the endpoint using the following:

```
https://jhub-tm252.ocl-dev.open.ac.uk/user/<your OCL ID>/code-server/proxy/8000/
```

Activity: Build a Basic HTTP Server

Go to the Module Website in a separate browser tab or window and access the OpenComputing Lab (OCL). You can find the link to access the OCL in the Resources section or in each week in the planner.

Having explored the basics of Python's HTTP server and request handling, complete the following activity.

In the OCL, click on the 25B TM252 Block 3 VCE.

1. Create a document and name it "http_server.py".
2. Save the document "http_server.py".
3. Add a comment to the file indicating your name and the date.
4. Modify the `handle_request` function shown in the example above to return a custom message (e.g., "Welcome to My Server!").
5. Change the response type to `text/html` and return an HTML message.
6. Add functionality to handle a custom endpoint (e.g. `/greeting`) that returns a different message in the form of "A message from Python". You should include both HTML and CSS to change the foreground and background colour of this message. Remember that you will need to change the content-type to 'text/html' if you have not already done so.
7. Save the script for future reference.

Week 19 - Deeper Exploration

The activities contained here are optional activities and have been included for those that wish to challenge their knowledge in Python development. Whilst the activities covered to this point cover necessary concepts to produce a useful web application, some of the concepts here are encountered far less frequently.

Variable Scopes

When discussing variables, we noted that they serve as locations to store data. We talked about dynamic typing, where variables can hold values of any type. While we can use type hints to suggest the intended data types for variables, it's important to understand that these hints are meant primarily for developers and tools like linters or IDEs—they do not affect how the interpreter handles the variables. We have also not yet covered the concept of variable scope.

The term 'variable scope' is used to denote the context in which a variable is assigned data and accessed within your script. Our previous scripts have seen the creation of variables, assigning data to them and their use later in these scripts. The vast majority of this creation and assignment has been done in what we would call the global scope. The term global scope is used to refer to the fact that a variable is accessible by the whole script.

In most cases, variables are used only within a small section of our script. This means that data stored in variables can be made accessible only within a limited part of the script. Such variables are called local variables—they exist and can be accessed only within a specific scope or "window" of the script, after which they are destroyed. This concept is most clearly seen when variables are declared inside functions.

```
def addnumbers(a,b):  
    result=0  
    result=a+b  
    print(result)  
  
print(result)
```

The code above defines a function using the `def` keyword, which takes two arguments, `a` and `b`. Inside the function, a variable `result` is declared and initially set to zero, then updated to store the sum of `a` and `b`. The function then prints the value of `result` before completing execution, successfully displaying the sum. However, when we try to print `result` outside the function, an error occurs. This happens because `result` is a local variable—its scope is limited to within the function, so the Python interpreter does not recognize it outside that scope.

```
File "/home/ou/TM252-25B/block_3/work/test.py", line 6, in <module>  
    print(result)  
    ~~~~~  
NameError: name 'result' is not defined
```

Advanced Data Structures

Arrays, lists, queues, and stacks are all data structures used to store collections of elements, but they differ in how they manage and access those elements. A list is an ordered, flexible collection that allows random access to elements and supports a wide range of operations like insertion, removal, and updating at any position. A queue follows the FIFO (First In, First Out) principle, where elements are processed in the order they are added, making it ideal for managing tasks or

requests in the order of arrival. A stack, on the other hand, follows the LIFO (Last In, First Out) principle, where the most recently added element is the first to be processed, commonly used for undo operations or backtracking. While lists are versatile for general-purpose use, queues and stacks are specialized structures suited for tasks that require strict order processing, with queues emphasizing the order of arrival and stacks emphasizing the reverse order.

In Python, a list is a versatile data structure that holds an ordered collection of elements. Lists are mutable, meaning their elements can be modified, added, or removed after creation. You can store any type of object inside a list, including other lists.

example usage of lists:

```
my_list = [1, 2, 3]
my_list.append(4) # Adds 4 to the list
print(my_list) # Outputs [1, 2, 3, 4]
my_list.remove(2) # Removes the value 2
print(my_list) # Outputs [1, 3, 4]
```

Lists are ideal when you need to store items in an ordered fashion, and access or modify them through index positions.

A queue is a data structure that follows the FIFO (First In, First Out) principle. The first element added to the queue will be the first one to be removed. Queues are often used in situations where tasks need to be processed in the order they arrive (such as handling requests in a web server).

Python's `queue.Queue` class can be used to implement a queue.

example usage of queues:

```
import queue # import queue to provide access to the queue module

q = queue.Queue()
q.put(1) # Adds 1 to the queue
q.put(2) # Adds 2 to the queue
print(q.get()) # Outputs 1 (removes 1 from the queue)
print(q.get()) # Outputs 2 (removes 2 from the queue)
```

Using a queue ensures that elements are processed in the order they were added, making it a useful structure for tasks like managing print jobs, processing orders, or handling concurrency.

A stack is a data structure that follows the LIFO (Last In, First Out) principle. The last element added to the stack will be the first one to be removed. Stacks are typically used in scenarios where you need to "undo" operations or navigate through previous states (e.g., browser history).

Python lists are commonly used to implement stacks with the `append()` and `pop()` methods. example usage of stacks:

```
stack = []
stack.append(1) # Pushes 1 onto the stack
stack.append(2) # Pushes 2 onto the stack
print(stack.pop()) # Outputs 2 (removes 2 from the stack)
print(stack.pop()) # Outputs 1 (removes 1 from the stack)
```

Stacks are ideal when you need to keep track of elements in reverse order or process items in a last in first out manner.

Ternary Operator

What is a Ternary Operator?

We have previously discussed conditionals and their use within our script to direct the flow of execution. In some cases, the use of consecutively nested conditionals can prove to be a little cumbersome and sometimes difficult to read. It is possible to condense some of these conditionals using a ternary operator in Python. A **ternary operator** is a shorthand conditional expression that allows you to assign a value based on a condition in a single line of code. It's a compact way to write an `if-else` statement.

Why Use the Ternary Operator?

Ternary operators are used when you need to choose between two values based on a condition but want to keep the code compact and readable. They can make your code more concise and easier to follow, especially for simple conditional logic. Ternary operators are typically used in situations like assigning values based on conditions, returning results from functions, or in list comprehensions.

In Python, the syntax for a ternary operator is:

```
value_if_true if condition else value_if_false
```

```
age = 20
status = "Adult" if age >= 18 else "Minor"
print(status) # Outputs: "Adult"
```

Note

Benefits of Ternary Operators

- **Concise Syntax:** They allow you to express conditional logic in a single line, making code shorter and easier to read for simple conditions.
- **Improved Readability:** When used correctly, they can enhance the readability of the code by reducing the verbosity of traditional if-else statements.
- **Cleaner Code:** Ternary operators help avoid unnecessary lines of code, particularly when you're performing simple conditional assignments.

Drawbacks of Ternary Operators

- **Reduced Readability for Complex Conditions:** For more complex conditions or logic, ternary operators can make the code harder to understand, especially for new developers or when nesting multiple operators.
- **Limited Use Cases:** Ternary operators are best suited for simple, clear conditional logic. They should be avoided for more complex conditional flows that require multiple actions or nested conditions.
- **Potential for Overuse:** If overused or used inappropriately, ternary operators can make the code less readable, defeating their purpose of simplification.

Warning

While it is technically possible to substitute almost all conditional statements to use this approach, it can make the code difficult to read. Using standard approaches to conditionals is far more readable as it is far clearer what condition is being evaluated.

Exploring Variable Scope, Data Structures, and Ternary Operators

Go to the Module Website in a separate browser tab or window and access the OpenComputing Lab (OCL). You can find the link to access the OCL in the Resources section or in each week in the planner.

Having explored the concepts of variable scope, data structures, and ternary operators, complete the following activity.

1. In the OCL, click on the 25B TM252 Block 3 VCE.
2. Create a document and name it "scope_and_structures.py".
3. Save the document "scope_and_structures.py".
4. Add a comment to the file indicating your name and the date.
5. Define a function called `calculate_sum` that takes two arguments and returns their sum. Assign a local variable inside the function and attempt to print it outside the function to observe variable scope behavior. The purpose of this is to demonstrate the lack of access to this variable.
6. Implement a stack using a list and add three items to it using the appropriate mechanism.
7. Implement a queue using Python's `queue.Queue()` and demonstrate adding five items to it using the appropriate method
8. Use a ternary operator to check if a number is positive or negative and print the appropriate message.
9. Run the script and verify your output.
10. Save the script for future reference.

An Introduction to Web Frameworks

We have already discussed web frameworks and compared their features, strengths, and use cases. Web frameworks provide a structured foundation for building web applications, offering tools and abstractions that simplify request handling, routing, templating, and other essential functionalities.

In the following activities our focus will shift specifically to Flask, a lightweight and flexible web framework for Python. Flask provides a minimalistic approach, allowing developers to build applications with only the necessary components while maintaining extensibility. In the next sections, we will explore how Flask enables us to implement and customize web applications efficiently, leveraging the skills developed in previous discussions.

The following activities have been designed to help you practice the skills discussed in this week's learning materials and provide you opportunity to build upon those concepts

An Introduction to Flask

As we have already seen, Flask is a lightweight web framework for Python that simplifies the process of building web applications. Flask is often used for building APIs, prototypes, and small-scale web applications, making it a great starting point for learning web development.

Flask enables you to define routes (URLs) and specify how the application should respond when those routes are accessed. Flask handles requests using Python's built-in HTTP handling capabilities, making it easier to create and test web applications locally.

Flask is not an in-built component of the Python language but an additional module that would need to be installed if you are using a personal machine for the activities here. We can include external modules in our scripts through the `import` and `from` keywords. `import` indicates the specific functionality we would like to include and `from` denotes the module where it is located. The following code extract offers an example of how this may be achieved.

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return 'A TM252 Flask application endpoint has been reached!'

if __name__ == '__main__':
    app.run(debug=True)
```

This example has one endpoint that can be accessed as the default route when sending a request to the server. This is seen in the following instruction:

```
@app.route('/')
```

This code associates the function `home()` with the route. To add another endpoint, simply define another route with `@app.route()` and attach a function as seen below:

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return 'Hello, World!'

@app.route('/welcome')
def greet():
    return 'Welcome to TM252!'

if __name__ == '__main__':
    app.run(debug=True)
```

Note

Observing the output of web applications using the VCE is a little different from running your Python script and seeing the output in the console. To see the visual output of your endpoints when using Flask, perform the following steps:

1. Ensure that you have installed the flask module by using the `pip` package manager. This can be achieved by typing the following command into the terminal window of the VCE. This command will install the latest version of the flask module in your environment. Without this package, you will not be able to complete the tasks for this week.

```
pip install flask
```

2. Ensure you have run the script using the terminal provided. This can be achieved by entering the following code into the terminal, replacing `<name of your file>` with the name of your Python script:

```
python3 -m flask --app '/home/ou/TM252-25B/block_3/<name and path to your file>.py' run
```

3. Once you have run this command, you may receive a notification at the bottom right of your screen asking if you would like to "Open in browser". Upon clicking this link, another tab will open showing the output of your default endpoint. If this bulletin does not appear, assuming you have completed step one, you can visit your application using the following link:

```
https://jhub-tm252.ocl-dev.open.ac.uk/user/<your OCL ID>/code-server/proxy/5000/
```

To access specific endpoints of your application, you need to append the endpoint name to the link provided in step 2. For example, for an endpoint named `getcolor`:

```
https://jhub-tm252.ocl-dev.open.ac.uk/user/<your OCL ID>/code-server/proxy/5000/getcolor
```

Note

The OCL ID can be found from the OCL home page

Benefits and Detriments of Using Flask

Note

Flask is suitable for small applications but may not be ideal for larger projects.

Benefits

- **Lightweight & Easy to Use** – Minimal setup, beginner-friendly.
- **Quick Development** – Ideal for rapid prototyping.
- **Built-in Development Server** – No extra configuration needed for local testing.
- **Customizable** – Provides flexibility compared to larger frameworks like Django.

Detriments

- **Not Suitable for Large Applications** – Can become difficult to manage as the project scales.
- **Development Server is Not Production-Ready** – Flask's built-in server lacks security and should not be used in production.
- **Limited Built-in Features** – No built-in authentication, admin panel, or ORM; requires third-party extensions.

Activity: Conditionals Practise

Go to the Module Website in a separate browser tab or window and from there access the OpenComputing Lab (OCL). You can find the link to access the OCL in the Resources section or in each week in the planner.

Having explored the ways in which web-frameworks can be employed, complete the following activity.

1. In the OCL click on the 25B TM252 Block 3 VCE.
2. Create a document and name it "`flask-framework.py`" 
3. Save the document `flask-framework.py` 
4. Add comment to the file to indicate the name and date for the file
5. Using the example above (found in the skeletons folder), add another endpoint to the application that returns the message "You have successfully reached the newly implemented endpoint". The route for this should be `/new-endpoint`
6. Add a further endpoint that returns the evaluation of `5 + 5`. This means that the endpoint should return the value `10`. You should use an appropriate endpoint name
7. Save the script for future reference

Implementing an echo-server

As we have seen, Flask is a lightweight framework with a low code footprint. This means that implementing the foundations of a useful application is far less complex.

The previous activity demonstrated how we could set up a rudimentary server using Flask and how to extend its functionality with additional endpoints.

It is possible to build upon this by implementing endpoints that can accept information in the request to that endpoint. By using Flask's request handling capabilities, we can create a simple echo server that accepts user input and sends it back as a response.

```
from flask import Flask, request

app = Flask(__name__)

@app.route('/echo')
def echo():
    message = request.args.get("message", "No message received")
    return f"You said: {message}"

if __name__ == '__main__':
    app.run(debug=True)
```

We can send the information to the endpoint using a standard URL format. When running Flask in the OCL environment, you should be able to access your web application through the OCL URL. To include the information, we use the `?` operator followed by the name of the data we are expecting; in this case `message` and the value of the data you want to send.

You may have noticed that there is an `f` before the string in this example. This tells Python to interpret any expressions inside curly braces as part of an f-string, which means the values of variables or expressions are automatically inserted into the string at runtime.

```
https://jhub-tm252.ocl-dev.open.ac.uk/user/<your OCL ID>/code-server/proxy/5000/echo?message=datatosend
```

If we were to call the above URL, the response would be as follows:

```
You said: datatosend
```

This example includes similar endpoints to those discussed in the previous activity in addition to a new endpoint that looks a little more complex. As with other concepts, we have introduced some further instructions here to achieve the task.

To handle the information being received, we have made use of the request functionality in the flask framework. To use this functionality we have to import the request class. This allows us to extract the information from the request and perform actions using it.

```
from flask import Flask, request
```

We then use the functionality of this class in the echo function

```
message = request.args.get("message", "No message received")
```

This line runs when the endpoint is accessed and checks the URL for a "message" parameter after the `?`. If found, its value is assigned to the variable `message`; otherwise, `message` defaults to "No message received." The code acts as a simple echo server, returning the value of `message` to the sender.

Activity: Implementing an Echo Server

Go to the Module Website in a separate browser tab or window and from there access the OpenComputing Lab (OCL). You can find the link to access the OCL in the Resources section or in each week in the planner.

Having explored the ways in which flask requests can be employed, complete the following activity.

1. In the OCL click on the 25B TM252 Block 3 VCE.
2. Create a document and name it `flask-echo.py` 
3. Save the document `flask-echo.py` 
4. Add comment to the file to indicate the name and date for the file
5. Using the example above (found in the skeletons folder), add another endpoint to the application that returns the message "You have successfully implemented an echo server" followed by your PID. You will find your PUD at the top of the page next to your name. The endpoint that you create should reflect the action being taken (for example 'echo')
 - Optional Extra Task
6. Save the script for future reference
7. you can terminate the Flask instance by selecting the terminal window and holding ctrl and pressing c.

Handling Parameters

In the previous section, we explored how Flask allows us to create an endpoint that accepts a single parameter using the `request.args.get()` method. This enabled us to extract data from the request and return it as a response, creating an echo server. However, real-world applications often require handling multiple pieces of data at the same time.

Flask provides a straightforward way to retrieve multiple parameters from a request, allowing us to pass more than one value within the request URL. By extending our previous example, we can modify our echo server to accept and process multiple parameters at once.

Consider the following example:

```
from flask import Flask, request

app = Flask(__name__)

@app.route('/extended-echo')
def echo():
    name = request.args.get("name", "Unknown")
    pid = request.args.get("pid", "Unknown")
    return f"Hello {name}, your Pid is {pid}"

if __name__ == '__main__':
    app.run(debug=True)
```

This new endpoint, '/extended-echo', demonstrates how we can retrieve multiple parameters from a request. We can call a URL with multiple parameters in the following way:

```
https://jhub-tm252.oc1.open.ac.uk/user/<your OCL ID>/code-server/proxy/5000/extended-echo?name=yourn
```

You may notice that we have introduced one further symbol in the URL; the `&` symbol. This symbol is used to separate the parameters given for the request allowing many pieces of information to be sent to the endpoint.

By implementing multiple parameters, we can make our Flask applications more interactive and capable of handling a wider range of requests. This concept is essential when building applications that require users to submit multiple pieces of data at once, such as registration forms, search queries, or API requests

Activity: Handling Multiple Parameters

Now that you have seen how Flask can handle multiple parameters, try the following task in the OpenComputing Lab (OCL):

1. Create a document and name it `flask-extended-echo.py` 
2. Save the document `flask-extended-echo.py` 
3. Add a comment to the file to indicate the name and date for the file
4. Add a new endpoint `/information` that accepts three parameters: `first_name`, `last_name`, and `course`.
5. Modify the response so that it returns a message in the following format: `Welcome [first_name] [last_name], you are enrolled in [course].`
6. Save and run the script. Test this using the console or the browser by visiting the appropriate URL.
7. Save the script for future reference.
8. to terminate the Flask application you can select the terminal window, hold Ctrl and press c

Request handling (GET and POST)

Now that you've seen how to handle parameters in Flask, let's take the next step and look at two of the most common types of HTTP requests that a Flask application might handle: GET and POST. These two methods are the foundation of web interactions.

The GET Request

In a GET request, data sent in this request is done so as part of the query string. This is similar to what we've already done when passing parameters using `request.args.get()`. GET requests are often used to get information from a server without modifying anything.

```
from flask import Flask, request

app = Flask(__name__)

@app.route('/welcome')
def responseMessage():
    name = request.args.get('name', 'Guest')
    return f"Welcome, {name}!"

if __name__ == '__main__':
    app.run(debug=True)
```

The POST Request

In contrast to a GET request, a POST request is used when you need to store or change information on the server. Unlike GET requests, POST requests do not append the data to the URL in a query string. POST data is included in the body of the request, making it more suitable for sensitive data and is often used for the transport of large amounts of information.

For instance, let's consider a form where a user submits their email and a message to a server through a contact form. Here's an example of how you could set this up in Flask using a POST request:

```
from flask import Flask, request

import requests #a further import is included here for redirection (this is used to automate the testing)

app = Flask(__name__)
@app.route('/contact', methods=['POST']) #note the use of the POST in the methods here
def contact():
    para1 = request.form.get('name', 'parameter one value') # if no parameter is passed, the default val
    para2 = request.form.get('message', 'parameter two value') #if no parameter given the default is sh
    return f"Received message from {para1}: {para2}"

@app.route('/test-contact', methods=['GET']) # or POST, depending on use
def test_redirect_contact():
    post_url = 'http://localhost:5000/contact'
    sample_data = {
        "name": "Tenzing Norgay",
        "message": "Greetings Traveller...."
    }
    try:
        response = requests.post(post_url, data=sample_data)
        return response.text
    except requests.exceptions.RequestException as e:
        return f"error{str(e)}", 500

if __name__ == '__main__':
    app.run(debug=True)
```

In this example, the `/contact` endpoint expects a POST request where the data is sent in the body of the request. The email and message are extracted from the form data using `request.form.get()`.

Sending requests using the POST method is a little more complicated than using a GET method here as we cannot just send the data using the URL. We can use HTML to implement a form that allows us to send a POST request to the application. Fortunately, you have already looked at HTML forms in previous weeks of block 2. An example form that can be used is demonstrated in the following extract:

```
<form action="<replace this with your OCL server URL>" method="POST">
  <input type="text" name="email" placeholder="Your email">
  <textarea name="message" placeholder="Your message"></textarea>
  <button type="submit">Submit</button>
</form>
```

Limiting Connections to Endpoints

You can control which HTTP methods are allowed for specific endpoints by using the methods parameter in the `@app.route` decorator. By default, Flask routes respond only to GET requests. However, if you want an endpoint to accept only certain

methods—such as POST for sending data—you can explicitly specify them. If a user tries to access this endpoint with a different method (like GET), Flask will return a “Method Not Allowed” error. This ensures that your endpoint only handles the types of requests you intend.

By completing these tasks, you’ll gain more experience in handling both GET and POST requests, which are essential for building dynamic web applications.

Activity: Understanding GET and POST Requests

Now that you’ve learned about GET and POST requests, try the following tasks:

1. Create a new Python file and name it `flask-post-example.py`.
2. Set up a POST endpoint that accepts a user’s username and password as parameters. You should name this endpoint `/userinfo`. This endpoint should return text in the following format:

```
f"Hello {username}, Welcome back!"
```

Instructions on how to test this are given from step 5.

3. Create a GET endpoint that accepts a Name and PID and responds with a greeting that includes these pieces of information. The endpoint should be called `/greet`.
4. Run your Flask application using the terminal commands outlined previously.
5. To test the POST endpoint from step 2, follow these substeps:
 - a. On your own computer, open a text editor and create a blank HTML file.
 - b. In this HTML file, add the basic HTML structure:

```
<!DOCTYPE html>
<html>
  <head>
    <title>POST Form</title>
  </head>
  <body>
    <form action="http://localhost:5000/userinfo" method="POST">
      <label for="username">Username:</label>
      <input type="text" id="username" name="username"><br><br>
      <label for="password">Password:</label>
      <input type="password" id="password" name="password"><br><br>
      <input type="submit" value="Submit">
    </form>
  </body>
</html>
```

- c. Make sure the `name` attributes of the form elements match the parameter names used in your Flask route.
 - d. Update the `action` attribute to reflect your endpoint’s URL path (e.g., `/userinfo`).
 - e. Save the HTML file.
6. Open the HTML file using your favourite web browser.
 7. When the page loads, enter information into both textboxes and click “Submit”. You should then see the greeting message from the Flask server.
 8. Ensure you save your Python script for future reference.

Handling Complex Data with JSON

Now that you've worked with GET and POST requests, it's time to dive into JSON data handling, a crucial concept in modern web development. JSON (JavaScript Object Notation) is a lightweight, text-based format used for storing and exchanging data. It's widely used for sending and receiving data between clients (like web browsers or mobile apps) and servers in a format that is easy for both humans and machines to read and write. What is JSON?

JSON is a data format that represents data as key-value pairs, much like dictionaries in Python. It is easy to work with because it can represent complex data structures, including nested objects and arrays. JSON is not limited to POST requests; it can be used in any HTTP request method (GET, POST, PUT, DELETE, etc.) that needs to send or receive structured data.

Here's a simple example of JSON data representing a student's details

```
{
  "name": "John",
  "PID": "Z11111",
  "enrolled_modules": [
    {
      "module_name": "TM111",
      "presentation": "24J"
    },
    {
      "module_name": "TM112",
      "presentation": "25B"
    }
  ]
}
```

In this example, we have a JSON object with key-value pairs. The **Enrolled Modules** component is a nested object containing its own key-value pairs for the module code and presentation of the module.

JSON in Flask POST Requests

Let's focus on POST requests first, since they're commonly used with JSON. When sending JSON in a POST request, the data is included in the body of the request, not the URL, allowing you to send structured data like the example above. This is more secure and flexible than sending data through form submissions as seen in previous activities for this block.

```

from flask import Flask, request, jsonify #Jsonify import included to handle conversion of data into the
import datetime
import requests # requests import included for managing the redirect here

app = Flask(__name__)

@app.route('/profile', methods=['POST'])
def profile():

    data = request.get_json()
    # Extract name and pid from the data
    name = data.get('name', 'Guest') #default to guest if no name is provided
    pid = data.get('pid', 'Unknown')
    return jsonify(f"Profile received for {name}, PID: {pid}")

#function used to call the '/profile' endpoint for testing, placeholder data is included in the post
# The placeholder data is in JSON format and is passed to the request. The response is also converted to
#JSON using the jsonify() function. This is then displayed in the browser.
@app.route('/test-profile', methods=['GET'])
def test_profile():
    # Make an actual POST request to your /profile endpoint
    post_url = 'http://localhost:5000/profile'
    # the use of json format data in this POST request is the key component here
    sample_data = {"name": "testuser", "pid": "Z1111111"}
    try:
        response = requests.post(post_url, json=sample_data)
        return jsonify(response.json())
    except requests.exceptions.RequestException as e:
        return jsonify({"error": str(e)}), 500

if __name__ == '__main__':
    app.run(debug=True)

```

In this example the `/profile` endpoint is set up to handle POST requests that send JSON data in the body. We use `request.get_json()` to parse the incoming JSON data into a Python dictionary. The name and pid are extracted from the data and returned as part of the response.

Sending JSON POST Requests

To send a JSON POST request, you can use external tools such as Postman or Curl, both of which are beyond the scope of this section. It is also possible to send a post request by using JavaScript.

Here's an example of how you may send a POST request with JSON in JavaScript:

```

fetch('http://localhost:5000/profile', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    name: 'John',
    age: 21
  })
})
.then(response => response.text())
.then(data => console.log(data));

```

In this example, the `fetch()` API sends a POST request with the JSON data in the body, specifying that the content is JSON with the `Content-Type: application/json` header

JSON in Other HTTP Methods

While we've focused on POST requests here, JSON can be used with any HTTP method that involves sending or receiving data. Much like the GET requests we have seen already might use JSON to return data from the server to the client.

Here's an example of using **JSON** in a **GET** request:

```
from flask import Flask, request

app = Flask(__name__)
@app.route('/user', methods=['GET'])
def user():
    user_data = {
        "name": "John",
        "pid": "Z11111"
    }
    # Flask will automatically convert this to JSON
    return user_data

if __name__ == '__main__':
    app.run(debug=True)
```

By completing these tasks, you'll gain hands-on experience with JSON requests and responses, which are essential for building APIs and working with modern web applications.

Activity: Working with JSON Requests

Now that you've learned a little about the basics of JSON and its use with different HTTP methods, try the following tasks:

1. Create a new Python file and name it `flask-json-example.py` .
2. Save the document `flask-json-example.py` .
3. Add a comment to the file to indicate the name and date for the file
4. Set up a POST endpoint `/add-book` that accepts a JSON object containing a `'book_name'` and a `'book_author'` key value pair and returns a message confirming the received data.
5. To test the endpoint, you can use either the javascript fetch functionality or create another test function as is shown in the example above. You could name this endpoint `/test-add-book`. This would have the same body as the `/test-profile` endpoint above but you would need to replace the call to `'http://localhost:5000/profile'`  with `'http://localhost:5000/add-book'`  and provide suitable values for the JSON data.
6. Modify your code to add more than two key-value pairs for the JSON object. For instance, a book will have attributes such as `'publication_year'` and `'price'` associated with it.
7. Create a GET endpoint `/get-book` that returns a books information in JSON format.
8. Save and run the script for future reference

By completing these tasks, you'll gain hands-on experience with JSON requests and responses, which are essential for building APIs and working with modern web applications.

Session Handling

Now that you've learned about handling JSON data and working with GET and POST requests, it's time to discuss Session Handling, an important concept for maintaining state in web applications. Sessions are essential for tracking user activity, keeping users logged in, and storing temporary data (like shopping carts or user preferences) across multiple requests.

A session allows you to store data that persists across multiple HTTP requests from the same user. Since HTTP is stateless (each request is independent of the others), session management helps keep track of user interactions with the application over time. The most common use of sessions is for user authentication—once a user logs in, their session is used to remember their identity, so they don't need to log in again on every request.

Flask uses a secure way of handling sessions based on a signed cookie mechanism. When you store session data in Flask, it is stored on the client side (in the browser) inside a cookie. While this data is not encrypted—meaning users can view its contents—it is cryptographically signed using the application's secret key. This signature ensures that the client cannot modify the data without invalidating the signature, helping to protect against tampering.

How Sessions Work in Flask

In Flask, you can use the session object to store and retrieve data across multiple requests. The data stored in the session is automatically sent back and forth between the client and server with each request.

Here's an example of how to handle sessions in Flask:

```

from flask import Flask, request, jsonify, session, redirect.
#session import has been added here to allow for flask session functionality,
#a further import in the form of redirect to handle true redirection to the root page following login
import datetime
import requests # requests imported to perform the redirect below

app = Flask(__name__)
app.secret_key = 'TM252SECRETKEY'

@app.route('/')
def home():
    if 'username' in session:
        username = session['username']
        return jsonify(f'Logged in as {username}')
    return jsonify('You are not logged in.')

@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')
    # For simplicity, assume a hard-coded check (you'd usually check a database)
    if username == 'student' and password == '53cur39455w0rd':
        # Store the username in the session
        session['username'] = username
        return redirect('/')
    return 'Invalid login credentials.'

@app.route('/test-login', methods=['GET'])
def test_login():
    # Make an actual POST request to your own /your-endpoint
    post_url = 'http://localhost:5000/login'
    sample_data = {"username" : "student", "password": "53cur39455w0rd"}
    try:
        response = requests.post(post_url, json=sample_data)
        return jsonify(response.json())
    except requests.exceptions.RequestException as e:
        return jsonify({"error": str(e)}), 500

if __name__ == '__main__':
    app.run(debug=True)

```

You can also limit access to certain routes based on the user's session data. For example, if the user is not logged in, you can redirect them to the login page.

Activity: Working with Sessions

Now that you understand the basics of session handling, try the following tasks:

1. Create a new Python file and name it `flask-session-example.py` .
2. Add a comment at the top indicating the filename and the date.
3. Set up a simple login system using sessions with endpoint `/login` taking three parameters; username, password and remember_me. Store the username in the session. Store the value of the "remember_me" option (e.g., `'remember_me' = True` if the parameter is 'yes', otherwise False). Display a welcome message including the username on the home page (`/`) when logged in.
4. Create a `/logout` endpoint that removes the username from the session using `session.pop()`. This will need to be used inside a request or Flask will present an error telling you as such.

```
session.pop('username', None).
```

5. Create a `/set-favourite-colour` endpoint that accepts a JSON key-value pair (`{"favourite_colour": "<replace_with_colour_value>"}`). Store the favourite colour in the session under `'favourite_colour'`.
6. Prevent users who are not logged in (i.e., no `'username'` in session) from accessing `/set-favourite-colour`. If a user tries to access it without being logged in, display a message informing them they need to log in first.
7. Save your script for future reference.

Persistent data storage

Now that you've learned how to manage user sessions, let's take the next step and introduce backing data storage. So far, we've been storing user data in sessions, which are suitable for temporary storage. However, session data is typically lost when the session expires or the server restarts (unless a server-side session store is used). In most real-world applications, user data, preferences, and records need to be stored persistently, which is where databases come in.

A database allows applications to efficiently store, retrieve, and manipulate data. For this section, we will use SQLite, a lightweight, file-based database system accessible via Python's built-in `sqlite3` module. SQLite is an excellent choice for small to medium applications because it requires no separate server setup and stores data in a simple file on disk.

Setting Up SQLite in Flask

Flask does not provide built-in database support but works well with SQLite through Python's standard library. When connecting to an SQLite database, the database file is created automatically if it doesn't exist. The crucial step is defining the database schema (tables) to store your data

The following code extract demonstrates how you can create a table for data storage in Python using SQLite.

```
import sqlite3
conn = sqlite3.connect('users.db')
cursor = conn.cursor()

cursor.execute(''' CREATE TABLE IF NOT EXISTS users ( id INTEGER PRIMARY KEY AUTOINCREMENT, username TEXT ) ''')

conn.commit()
conn.close()
```

Storing User Data in Flask

Now that we have a database set up, let's integrate it with Flask to allow user registration and login using stored credentials.

```

from flask import Flask, request, session, redirect, url_for, jsonify
import sqlite3. #a further import is included to add functionality for database management in SQLite3, t
import requests

app = Flask(__name__)

app.secret_key = 'TM252SECRETKEY'
DATABASE_NAME='test.db'

def init_db():    # function definition to initialise the database
    conn = sqlite3.connect(DATABASE_NAME) # connection uses the database name to connect
    cursor = conn.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS users (
            user_id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT UNIQUE NOT NULL,
            password TEXT NOT NULL
        )
    ''') #SQL statement to create a table in the event it doesn't exist. This statement will create a ta

#function to permit the registration of a user. This stores the username and password into the database.
#Note that this is for illustration only and would not be sensible on a real server. storage of a passwo
#be a far more sensible option for this application
@app.route('/register_user', methods=['POST'])
def register_user():
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')

    if not username or not password:
        return jsonify("Missing username or password", 400)

    conn = sqlite3.connect(DATABASE_NAME)
    cursor = conn.cursor()
    try:
        cursor.execute("INSERT INTO users (username, password) VALUES (?, ?)", (username, password))
        conn.commit()
        return jsonify("successful registration")
    except sqlite3.IntegrityError:
        return jsonify("Unsuccessful registration") # user already exists
    finally:
        conn.close()

#Test method included to allow for post on the OCL server
@app.route('/test-register_user', methods=['GET'])
def test_register_user():
    # Make an actual POST request to your own /your-endpoint
    post_url = 'http://localhost:5000/register_user'
    sample_data = {"username" : "student", "password": "53cur39455w0rd"}
    try:
        response = requests.post(post_url, json=sample_data)
        return jsonify(response.json())
    except requests.exceptions.RequestException as e:
        return jsonify({"error": str(e)}), 500

init_db()
if __name__ == '__main__':
    app.run(debug=True)

```

Retrieve a user's details on login

```
from flask import Flask, request, jsonify
import sqlite3
import datetime
import requests

app = Flask(__name__)

app.secret_key = 'TM252SECRETKEY'
DATABASE_NAME = 'test.db'

@app.route('/test-login', methods=['GET'])
def test_login():
    # Make an actual POST request to your own /your-endpoint
    post_url = 'http://localhost:5000/login'
    sample_data = {"username": "student", "password": "53cur3"}
    try:
        response = requests.post(post_url, json=sample_data)
        return jsonify(response.json())
    except requests.exceptions.RequestException as e:
        return jsonify({"error": str(e)}), 500

def init_db():
    conn = sqlite3.connect(DATABASE_NAME)
    cursor = conn.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS users (
            user_id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT UNIQUE NOT NULL,
            password TEXT NOT NULL
        )
    ''')
    conn.close()

@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')

    if not username:
        return jsonify("Missing username", 400)

    conn = sqlite3.connect(DATABASE_NAME)
    cursor = conn.cursor()
    cursor.execute("SELECT user_id, username FROM users WHERE username = ?", (username,))
    user = cursor.fetchone()
    conn.close()

    if user:
        user_id, username = user
        return jsonify({'user_id': user_id, 'username': username})
    else:
        return jsonify("User not found", 404)

init_db()

if __name__ == '__main__':
    app.run(debug=True)
```

Testing the Application

To test this application, you can copy the examples above to the OCL and run in the same way as the other flask applications within this block.

Activity: Storing and Retrieving User Data

Now that you have covered the basics of using SQLite in Flask, try these tasks:

By completing these tasks, you'll get hands-on experience in integrating databases with Flask, making your applications more functional and data-driven.

1. Create a new Python file and name it `flask-database-example.py`.
2. Add a comment to indicate the filename and date.
3. Copy the `/register_user` endpoint from above. Add a basic check for insecure passwords and return an "insufficient complexity" message if the password resembles `"password1"`.
4. Implement a GET endpoint `/get_users` that returns a list of registered users in JSON format.
5. Modify the login system to store the login timestamp in the database. You will need to add another field to the table for this. To do this, either:

- Delete the existing database and update the `init_db` function to include an additional field in the `CREATE TABLE` command:

```
last_login TEXT
```

- Or update the table using the `ALTER TABLE` command for SQLite3:

```
ALTER TABLE users ADD COLUMN last_login TEXT;
```

6. Run the script to test user registration and authentication endpoints.
7. Save the script for future reference.

Web Services Overview

Now that you've learnt how to store and retrieve data using SQLite, let's take the next step and explore how to share this data with other applications. Storing data in a database is useful, but in modern web development, applications often need to communicate with other systems—this is where web services come in.

A web service is a way for applications to exchange data over the internet using a commonly used format, such as JSON or XML. Web services allow different applications—regardless of programming language or platform—to communicate with one another.

Why Use Web Services?

Web services are used for many reasons, such as:

1. Allowing different applications to exchange data.

2. Enabling mobile apps to fetch data from a web server.
3. Supporting third-party integrations, such as connecting an e-commerce site to a payment gateway.
4. Making data publicly accessible, like an API that provides weather forecasts.

However, web services also have some drawbacks:

1. They require network access, meaning they won't work offline.
2. Poorly designed services may have security vulnerabilities.
3. They can slow down an application if not properly optimised.

RESTful Services

When working with web services, you'll often hear the term RESTful API. REST stands for Representational State Transfer, and it is a set of architectural principles for designing networked applications. A RESTful API follows these principles to enable efficient and scalable communication between clients (like web browsers or mobile apps) and servers.

A RESTful API uses HTTP methods to interact with resources. Some of which we have already covered this week:

- **GET**: Retrieve data.
- **POST**: Submit data to the server, typically to create a new resource.
- **PUT**: Replace an existing resource or create a new one at a specific URL.
- **DELETE**: Remove data, making the intent of the request clear.

Each request targets a specific resource (like `/users`) and follows a standardised structure, making RESTful APIs easy to understand and use. By following REST principles, web services remain simple, efficient, and scalable, making them the preferred choice for modern web development.

RESTful services

In Flask, we can create a RESTful web service using routes that return data in JSON format. This is the same as putting all of the skills we have discussed into a single script. Below is a simple example of a rudimentary web service that provides user data from an SQLite database.

```

from flask import Flask, jsonify, request
import sqlite3

app = Flask(__name__)

DATABASE_NAME = 'test.db' # Use this database name consistently

def init_db():
    """
    Initialise the database and create the users table if it doesn't exist.
    """
    conn = sqlite3.connect(DATABASE_NAME)
    cursor = conn.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS users (
            user_id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT UNIQUE NOT NULL,
            password TEXT NOT NULL
        )
    ''')
    conn.commit() # Commit changes after table creation
    #This will add a record to the database to ensure there is a user to retrieve
    #Used for illustration only - the use of the UNIQUE identifier for the field will prevent entry of t
    cursor.execute('INSERT INTO users (username,password) VALUES ("tony","53cr3t9455w0rd")')
    conn.commit()
    conn.close()

def get_users():
    """
    Retrieve all users from the database.
    """
    conn = sqlite3.connect(DATABASE_NAME)
    cursor = conn.cursor()
    cursor.execute("SELECT user_id, username FROM users")
    users = cursor.fetchall()
    conn.close()
    # Return list of dicts without passwords
    return [{"user_id": user[0], "username": user[1]} for user in users]

@app.route('/users', methods=['GET'])
def users():
    """
    API endpoint to get all users.
    """
    return jsonify(get_users())

@app.route('/users/<int:user_id>', methods=['GET'])
def get_user(user_id):
    """
    API endpoint to get a specific user by ID.
    """
    conn = sqlite3.connect(DATABASE_NAME) # Use the consistent database name here
    cursor = conn.cursor()
    cursor.execute("SELECT user_id, username FROM users WHERE user_id = ?", (user_id,)) # Use correct c
    user = cursor.fetchone()
    conn.close()

    if user:
        return jsonify({"user_id": user[0], "username": user[1]}) # Return user info without password
    return jsonify({"error": "User not found"}), 404

init_db() # Ensure the database and table are created before the app runs

```

```
if __name__ == '__main__':  
    app.run(debug=True)
```

A simple API like this can be extended by adding more HTTP methods through the same endpoint format used in previous sections of this weeks activities. Implementing web applications in this way allows us to use the functionality provided by the application in devices that may not use a traditional web-browser interface.

- POST requests to allow users to be added via the API.
- PUT requests to update user details.
- DELETE requests to remove users.

Activity: Building a Web Service in Flask

Go to the Module Website in a separate browser tab or window and from there access the OpenComputing Lab (OCL). You can find the link to access the OCL in the Resources section or in each week in the planner.

Now that you've seen how to create a basic web service, try the following tasks:

By completing these tasks, you'll gain hands-on experience in building RESTful web services, an essential skill in modern web development.

1. Create a document and name it "restful_service.py"
2. Save the document "restful_service.py"
3. Add a comment to the file to indicate the name and date for the file
4. Modify the '/users' endpoint to return only two users.
use the **LIMIT** keyword to restrict the number of rows returned.

```
SELECT user_id, username FROM users LIMIT 2;
```

5. Add a '/delete_user' endpoint that removes a user from the database that takes the id as a parameter.
Use the SQL **DELETE** keyword to remove the user by **user_id**.

```
DELETE FROM users WHERE user_id = ?;
```

6. Test each endpoint by starting the Flask application in the OCL and visiting the appropriate endpoints.
7. Save your script for future reference.

Week 20 - Further Exploration

SOAP vs REST: Understanding the Differences

Now that we've explored RESTful services, it's important to understand how they compare to another common web service protocol: **SOAP (Simple Object Access Protocol)**. Both REST and SOAP are used for communication between applications, but they differ in their structure, implementation, and use cases.

Key Differences Between SOAP and REST

Feature	SOAP	REST
Protocol	Uses XML-based messaging	Uses standard HTTP methods
Data Format	Only XML	JSON, XML, HTML, or others
Complexity	More complex, requires strict standards (WS-* specs)	Simpler, more flexible
Performance	Slower due to XML overhead	Faster due to lightweight JSON
Security	Built-in WS-Security standards for encryption, signing, authentication	Relies on HTTPS, OAuth, JWT
Use Case	Banking, enterprise applications requiring transactional reliability	Web APIs, mobile applications

SOAP is often used in scenarios where **strict security and transactional reliability** are required, such as banking and financial services. REST, on the other hand, is **lightweight and flexible**, making it the preferred choice for web and mobile applications.

Security in Web Services

Web services must be **secure** to prevent unauthorized access, data breaches, and attacks like **man-in-the-middle (MITM) attacks, SQL injection, and cross-site scripting (XSS)**. There are several ways to protect web services, including authentication, encryption, and validation techniques.

JSON Web Tokens (JWT)

One common way to secure RESTful APIs is by using **JSON Web Tokens (JWTs)**. A JWT is a compact, self-contained token that securely transmits information between parties.

How JWT Works

1. The client authenticates and receives a **JWT token** from the server.
2. The client includes the JWT in the **Authorization** header of future requests (usually prefixed by "Bearer ").
3. The server verifies the JWT and grants access if it is valid.

Benefits of JWT

- **Stateless authentication** – No need for session storage on the server.
- **Compact and efficient** – Can be sent in HTTP headers.
- **Secure** – Uses cryptographic signing to prevent tampering.

Drawbacks of JWT

- **Cannot be easily revoked** – Once issued, JWTs remain valid until expiration unless additional measures like blacklists are implemented.
- **Larger token size** – Can increase request overhead compared to session cookies.

Example: Protecting an API with JWT in Flask

```
from flask import Flask, request, jsonify #import of jsonify to convert data for message transfer
import datetime
from flask_cors import CORS
import requests #requests is used for testing in this script. we call the login functionality from '/test_login'
import jwt #Imports the PyJWT functionality to generate the JWT token

app = Flask(__name__)
app.config['SECRET_KEY'] = 'supersecretkey' # Set your secret key here

CORS(app) # This is used for testing only (as we are using protected OCL)

# This is a test endpoint included so that you can call the login POST request
@app.route('/test_login', methods=['GET'])
def test_login():
    # Make an actual POST request to your own /your-endpoint
    post_url = 'http://localhost:5000/login'
    sample_data = {"username": "admin", "password": "53cur3"}. # this is an example set of post data included
    try:
        response = requests.post(post_url, json=sample_data) # here we start the post request to the '/login' endpoint
        return jsonify(response.json()) # we then return this response as json to the browser
    except requests.exceptions.RequestException as e:
        return jsonify({"error": str(e)}), 500

# Generate a JWT token
@app.route('/login', methods=['POST'])
def login():
    username = request.json.get('username')
    password = request.json.get('password')
    if username == "admin" and password == "53cur3": #determines if valid username and password combination
        #uses the jwt functionality to generate the jwt token
        token = jwt.encode({'user': username, 'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=1)})
        return jsonify({'token': token})
    return jsonify({'message': 'Invalid credentials'}), 401

# Protected route - this is used to make requests of from the server with the associated token
@app.route('/protected', methods=['GET'])
def protected():
    token = request.headers.get('Authorization') # This request is expecting that you have added the token to the headers
    if not token:
        return jsonify({'message': 'Token is missing!'}), 403
    try:
        data = jwt.decode(token, app.config['SECRET_KEY'], algorithms=["HS256"]) #attempts to decode the token
        return jsonify({'message': f'Welcome {data["user"]}! This is a protected route.'}) # if the token is valid
    except:
        return jsonify({'message': 'Token is invalid!'}), 403

if __name__ == '__main__':
    app.run(debug=True)
```

Note

to run the above code, you will need to install the PyJWT,requests and flask-cors modules using pip. This can be achieved by typing the following command into the terminal window of the VCE. This command will install the latest version of the flask module in your environment. Without this package, you will not be able to complete the tasks for this week.

1. Ensure that you have installed the PyJWT,flask-cors and requests modules by using the `pip` package manager. This can be achieved by typing the following command into the terminal window of the VCE. This command will install the latest version of the flask module in your environment. Without this package, you will not be able to complete the tasks for this week.

```
pip install PyJWT
pip install requests
pip install flask-cors
```

2. Ensure you have run the script using the terminal provided. This can be achieved by entering the following code into the terminal, replacing `<name of your file>` with the name of your Python script:

```
python3 -m flask --app 'https://jhub-tm252.ocl.open.ac.uk/user/<your OCL ID>/code-server/proxy/5000/'
```

3. Once you have run this command, you may receive a notification at the bottom right of your screen asking if you would like to "Open in browser". Upon clicking this link, another tab will open showing the output of your default endpoint. If this bulletin does not appear, assuming you have completed step one, you can visit your application using the following link:

```
https://jhub-tm252.ocl-dev.open.ac.uk/user/<your OCL ID>/code-server/proxy/5000/
```

While JWT helps secure APIs, additional security practices should also be followed:

- Use HTTPS – Encrypts data between client and server.
- Implement Rate Limiting – Prevents excessive API requests (DDoS attacks).
- Validate Input Data – Prevents SQL injection and XSS attacks.
- Use API Keys or OAuth – Adds an extra layer of authentication.
- Log and Monitor Requests – Detects unusual activity or potential threats.

By implementing these security measures, web services can protect sensitive data and ensure secure communication between clients and servers.

Activity: Securing Your API

Go to the Module Website in a separate browser tab or window and from there access the OpenComputing Lab (OCL). You can find the link to access the OCL in the Resources section or in each week in the planner.

Now that you've learned about securing web services with JWT, try the following tasks:

In the OCL click on the 25B TM252 Block 3 VCE.

1. Create a new Python file and name it `secure_api.py`.
2. Save the document `secure_api.py`.
3. Add comments to indicate the name and date of the file.
4. Implement JWT authentication for a Flask API using the above example. you should include a `/login` endpoint a `/get-account-details` endpoint and a `/logout` endpoint.
5. Using the code above copy and modify the `/protected` route to display the logged-in user's name.
6. Experiment with token expiration times and error handling.
7. Save your script for future reference

Errata

- **30.05.2025**
 - [An Introduction to Flask](#)
Updated to provide clarity on running flask in VCE
- **29.05.2025**
 - [Python variables](#)
variable labels changes from `name` and `age` to `daffodilCount`
 - [Iteration in Python](#)
filename updates from `script_conditionals.py` to `script_iteration.py` (step 2)
- **17.05.2025**
 - [Environment Overview](#)
Where Next content removed

Accessibility Statement for TM252 Block 3 Activities

The Open University is committed to making its websites and mobile applications accessible, in accordance with the Public Sector Bodies (Websites and Mobile Applications) (No. 2) Accessibility Regulations 2018.

This accessibility statement applies to the TM252 Block 3 Activities.

We want as many people as possible to be able to use our websites and mobile apps, and accessibility is an essential part of our mission. On our [Accessibility Hub](#) , you'll find everything you need to answer any questions you have about accessibility, whether you're a student or a member of staff.

To adapt the content to your needs or preferences you should be able to:

- Change colours, contrasts levels and fonts.
- Resize text up to 200% without impact on the functionality of the website.
- Zoom in up to 400% without loss of information or functionality.
- Navigate the website using just a keyboard.
 - tab to 'Skip to content' links at the top of the page to jump over repetitive information to the main content.
 - tab through the content; the current location will be indicated by a clear visual change.
 - control the embedded media player to play audio and video materials.
- Use a screen reader (e.g. JAWs, NVDA) to:
 - listen to the content of web pages and use any functionality on the page.
 - list the headings and subheadings in the page and then jump to their location on the page.
 - bring up a list of meaningful links on the page.
- Use transcripts or closed captions with most audio and video materials.
- Download learning materials in alternative formats (e.g. Word document, PDF, ePub, Kindle eBook).
- If you have a print disability we provide [SensusAccess](#)  to students, which is an automated service that converts files from one format to another, for example, PDF to text, audio, Word or Braille.
- [AbilityNet](#)  also provides advice on making your device easier to use if you have a disability.

Compliance status

This website is fully compliant with the Web Content Accessibility Guidelines version 2.2 AA standard.

Preparation of this accessibility statement

This statement was prepared on 26.06.2023.

The statement was last reviewed on 26.06.2023.

The website was last tested on 26.06.2023.

The website was tested using a combination of automatic accessibility checking tools as part of the website's standard development process.

Feedback and contact information

If you find that a certain section of our website is not accessible and you can't get access to the information that you need please use the Open [University Accessibility Feedback Form](#)  to request support and we will ensure that you are provided with the information you require. You will need to provide your contact details and Personal Identifier if you are a student so we can get back to you. You should expect to hear back from us within 5 working days.

The OU is very experienced in meeting accessibility needs for our students. In many cases we are able to provide module and other study support materials in alternative formats for students who indicate a need for this when completing a [Disability Support Form](#) .

In addition, some module materials are available in different formats and can be downloaded from module websites. Students can contact their [Student Support Team](#)  for advice.

If you are a student, or someone who has had contact with the University before, and have a complaint about the accessibility of our websites, you should raise a complaint via the [complaints and appeals process](#) .

Enforcement procedure

The Equality and Human Rights Commission (EHRC) is responsible for enforcing the Public Sector Bodies (Websites and Mobile Applications) (No. 2) Accessibility Regulations 2018 (the 'accessibility regulations').

If you are based in the UK, and you're not happy with how we respond to your complaint, contact the [Equality Advisory and Support Service \(EASS\)](#) . If you are based in Northern Ireland, and you're not happy with how we respond to your complaint, contact the [Equalities Commission for Northern Ireland \(ECNI\)](#) .